Module 3: Parsing

Recursive Descent

AKA How to implement a perfectly acceptable parser.

1.

Recursive Descent

In order to understand recursive descent,
you must first understand recursive descent.

Recursive Descent

Definitions

Recursive Descent is an approach where each rule becomes a function.

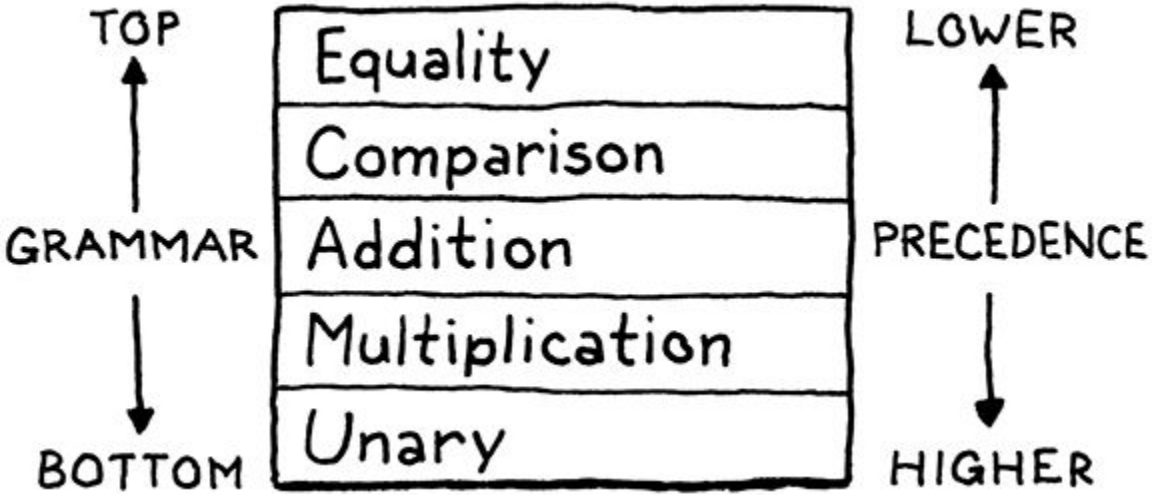
- Referencing another rule is equivalent to calling that function.
  - Nested expressions -> recursion
- Operator precedence is managed through the *call stack*.
  - Higher functions have priority for continuing expressions.

Grammar Style

The style of our grammar should be written in a way that works for recursive descent:

- LL(~1): We can determine which rule to parse based on 1 (*maybe* 2) tokens of lookahead.
  - Left recursion is a huge nope!
  - We *definitely* don't want backtracking.
- Operator precedence should be encoded as rules in the grammar.
  - Other approaches (e.g. Pratt parsing) exist, but are a step beyond recursive descent.

```
2 * x + y == z
z == y + x * 2
```



Operator Precedence in Recursive Descent

Module 3: Parsing

Recursive Descent

AKA How to implement a perfectly acceptable parser.

Module 3: Parsing

Parser Introduction

One down, four to go!

Slides Carnival

Free templates for all your presentation needs

For PowerPoint and Google Slides

100% free for personal or commercial use

Ready to use, professional and customizable

Blow your audience away with attractive visuals